CHANDIGARH UNIVERSITY

Department of Information Technology

A PROJECT REPORT

on

Voting-Poll System

Submitted by

Achintya Srivastava

Voting App Architecture Report

1. Project Overview

The Voting App is a full-stack web application designed to allow users to create polls, vote on them, and view results. The system is split into two main components:

- **Backend:** A Java-based REST API built with Spring Boot.
- **Frontend:** A Single Page Application (SPA) built with React and Vite.

2. Technology Stack & Dependencies

Backend Technologies

The backend is built as a robust and scalable Spring Boot application.

- **Core Framework:** Spring Boot 4.0.3 (using Java 21)
- **Database:** MySQL
- **Data Access:** Spring Data JPA
- **API Architecture:** RESTful services using Spring Web MVC

Backend Dependencies (pom.xml)

- spring-boot-starter-webmvc: Provides tools to build web, including RESTful, applications using Spring MVC.
- spring-boot-starter-data-jpa: Adds support for Java Persistence API to interact with the database.
- mysql-connector-j: The JDBC driver for MySQL to allow the Java app to connect to the database.
- lombok: A library used to reduce boilerplate code (such as getters, setters, constructors) via annotations.
- spring-boot-starter-data-jpa-test & spring-boot-starter-webmvc-test: Used for unit and integration testing.

Frontend Technologies

The frontend is a modern React application managed and built using Vite.

- **Core Library:** React 19.2.0

- **Build Tool:** Vite 7.3.1

Frontend Dependencies (package.json)

- **Dependencies:** react, react-dom
- **DevDependencies:** vite, @vitejs/plugin-react, eslint and various related plugins for code linting and standardizing structure.

---

3. Backend Architecture

The backend follows the classic Multi-Tier Architecture (Controller-Service-Repository pattern):

3.1 Entities (Data Models)

Located in com.votingApp.votingApp.entity, these represent the database tables:

- **User.java**: Represents a user in the system (used for authentication).
- **Poll.java**: Represents a poll containing the question and its options.
- **OptionVote.java**: Represents the individual options for a poll and their corresponding vote counts.

3.2 Repositories (Data Access Layer)

Located in com.votingApp.votingApp.Repository, these interfaces extend Spring Data JPA's repository interfaces to provide built-in CRUD operations and custom query capabilities to the database.

3.3 Services (Business Logic Layer)

Located in com.votingApp.votingApp.service, these classes contain the core business logic.

- **UserService.java**: Handles logic for user sign-up, login, and authentication.
- **pollService.java**: Handles creating polls, retrieving polls, and the logic to increment votes on specific poll options.

3.4 Controllers (Presentation/API Layer)

Located in com.votingApp.votingApp.controller, these classes expose the REST API endpoints to the frontend.

- **AuthController.java**:
    - POST /api/auth/signup: Registers a new user.

    - POST /api/auth/login: Authenticates an existing user.
- **pollController.java**:
    - POST /api/polls: Creates a new poll.
    - GET /api/polls: Retrieves all available polls.
    - GET /api/polls/{id}: Retrieves a specific poll by its ID.
    - POST /api/polls/vote: Submits a vote for a specific option on a specific poll.

NOTE

Both controllers utilize the @CrossOrigin(origins = "http://localhost:5173") annotation to explicitly allow the React development server to communicate with the Spring Boot backend without encountering CORS errors.

---

4. Frontend Architecture

The frontend is a component-based application structured inside the src directory.

4.1 Component Structure

Located in src/components:

- **Auth.jsx**: Handles the user interface for Login and Signup.
- **CreatePollForm.jsx**: Provides a form UI for authenticated users to create new polls.
- **PollList.jsx**: Renders the list of available polls and allows users to interact/vote on them.
- **App.jsx**: The main entry point component that brings together the authentication flow and the main application dashboard.

4.2 API Integration

Located in src/api, these files isolate the network requests to keep the UI components clean:

- **auth.js**: Contains asynchronous functions to call the backend /api/auth endpoints.
- **polls.js**: Contains asynchronous functions to fetch, create, and vote on polls via the backend /api/polls endpoints.

4.3 Styling

- **App.css** & **index.css**: Contain vanilla CSS rules to format the application, maintaining a clear separation of style and structure.

---

5. Summary

The Voting App implements a clean and standard full-stack architecture. The backend isolates database logic, business rules, and HTTP endpoints perfectly via the Controller-Service-Repository pattern. The frontend is modularized effectively, separating network calls (api/ folder) from the presentation layer (components/ folder). This separation of concerns ensures the project is maintainable, scalable, and easy to navigate.